

EduSaaS Coding Assessment Platform

Architecture, Workflow, API, Docker Sandbox, Security, Database and Setup Documentation

Prepared from the supplied project documentation

1. System Overview

The EduSaaS Coding Assessment Platform is an enterprise-grade automated grading and remote code execution system. It supports real-time multi-language evaluation (Python, C++, Java) inside isolated, resource-constrained Docker containers. The platform automates candidate session lifecycles, runs database-backed test validation (sample and hidden test cases), calculates scores, and records execution metrics.

2. Project Directory Structure

```
edusaas-compiler/
├── config/
│   └── db.js
├── controllers/
│   └── codeController.js
├── routes/
│   └── codeRoutes.js
├── services/
│   └── dockerService.js
├── sql/
│   └── schema.sql
├── .env
├── .gitignore
├── package.json
└── server.js
```

File Inventory

- `server.js`

Bootstraps the Express application, registers CORS and parsing middleware, and starts the HTTP server.

Runs container engine health checks during boot to guarantee execution readiness.

- `package.json`

Defines project dependencies, Node.js scripts, and package metadata.

Locks runtime driver versions including pg (PostgreSQL client) and Express utilities.

- `.gitignore`

Excludes volatile files like node_modules/, system logs, and environment variables from Git.

Prevents accidental leaks of internal credentials and transient compile artifacts.

- ``.env``

Contains external configuration keys including database connection strings, ports, and limits.

Maintains complete separation between application code and deployment environments.

- ``config/db.js``

Initializes and exports the PostgreSQL connection pool using node-postgres (pg).

Manages concurrent connection lifecycles, query pooling, and reconnection logic.

- ``controllers/codeController.js``

Implements the assessment logic: session creation, test running, submission scoring, and final grading.

Coordinates database transactions with sandbox execution calls and normalizes test outputs.

- ``routes/codeRoutes.js``

Exposes REST API endpoints and routes incoming HTTP requests to their controller handlers.

Serves as the primary routing module mounted under the `/api` prefix.

- ``services/dockerService.js``

Spawns sandboxed Docker containers to compile and execute Python, Java, and C++ code.

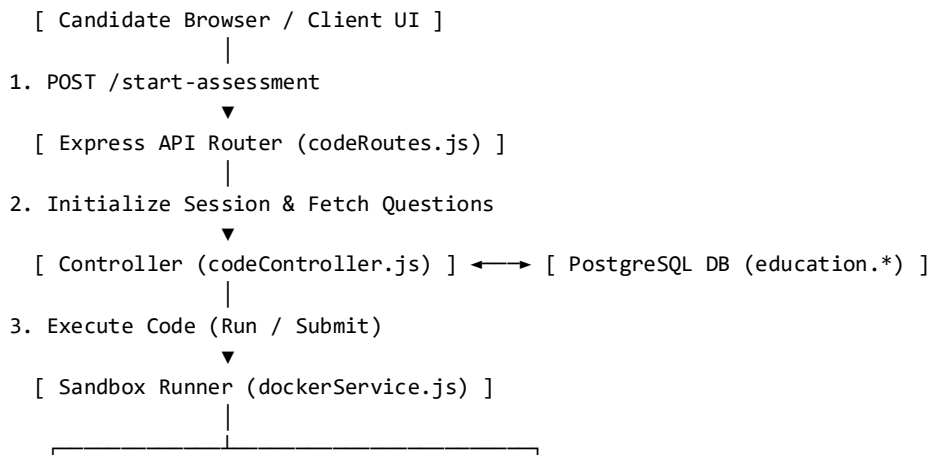
Enforces memory/CPU quotas, read-only volume bindings, process limits, and execution timeouts.

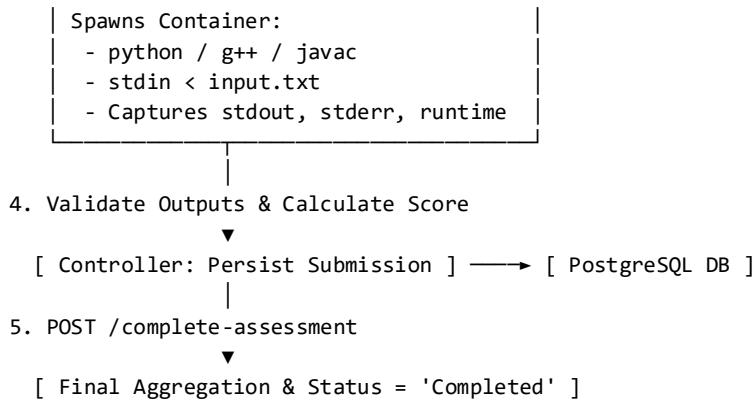
- ``sql/schema.sql``

Defines PostgreSQL DDL schemas for questions, test cases, sessions, and submissions.

Establishes foreign keys, primary indexes, column constraints, and relational schemas.

3. End-to-End System Workflow





Execution Lifecycle Stages

Stage 1: Assessment Initialization (`POST /api/start-assessment`)

1. The client passes `userId` to start an assessment.
2. The controller verifies whether an active session already exists for the user. If not, it creates a new entry in `education.user\_coding\_sessions` with status `In Progress`.
3. The server retrieves the assigned questions and their public sample test cases (`is\_hidden = false`) from `education.coding\_test\_cases`.
4. The client receives the session metadata and question list to populate the IDE interface.

Stage 2: Code Execution & Sample Testing (`POST /api/run-code`)

5. The candidate writes code in the editor and clicks "Run".
6. If `questionId` is provided:
 - The backend queries all public sample test cases for that question.
 - For each test case, the code is passed to `dockerService.js` along with the test input.
7. If custom input is provided, it executes a single run with the specified input.
8. The controller normalizes string outputs (`\r\n` \$\rightarrow\$ `\n`, `.trim()`), checks for exact match against expected output, and returns an itemized pass/fail breakdown.

Stage 3: Formal Submission & Grading (`POST /api/submit-code`)

9. The candidate submits their final solution for a specific question.
10. The backend fetches all test cases (both public samples and hidden benchmark cases).
11. The controller executes each test case sequentially inside isolated containers:

- Correct output: Marked as `Passed`.
- Mismatch: Marked as `Wrong Answer`.
- Timeout (\$> 8000\text{ ms}\$): Marked as `Time Limit Exceeded`.
- Syntax/Runtime Crash: Marked as `Runtime Error`.

12. The score is calculated proportionally:

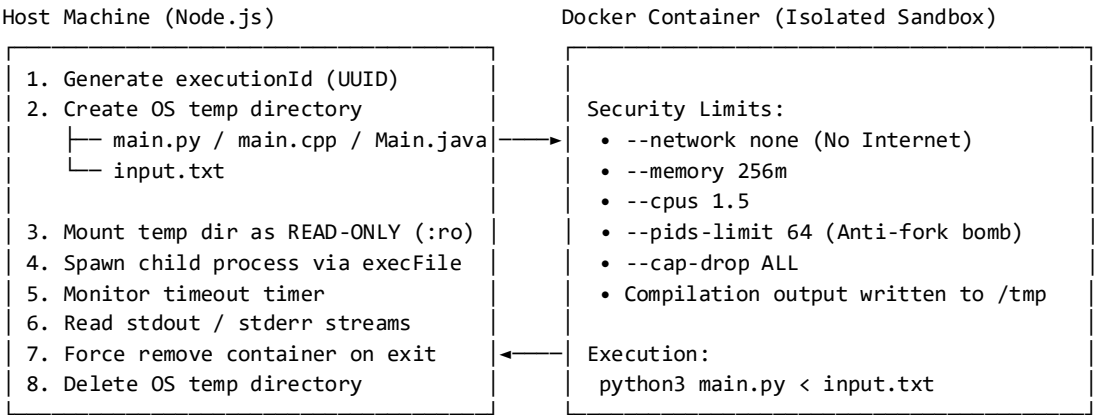
$$\text{Score} = \left(\frac{\text{Passed Test Cases}}{\text{Total Test Cases}} \right) \times \text{Total Question Marks}$$

13. An entry is inserted into `education.coding\_submissions` logging the source code, score, execution time, and individual test results.

Stage 4: Assessment Finalization (POST /api/complete-assessment)

- 14. After all questions are attempted, the client triggers the final completion call.
- 15. The controller opens a database transaction (BEGIN), queries the highest score achieved per question for the given sessionId, and computes the sum.
- 16. The session status in education.user_coding_sessions is updated to 'Completed', storing total_score and questions_completed.
- 17. The transaction commits (COMMIT) and returns the final assessment report.

4. Docker Sandbox Execution Architecture



Language-Specific Execution Commands

Language	**Target File**	**Sandbox Command**
Python	`main.py`	`python3 main.py < input.txt`
C++	`main.cpp`	`g++ main.cpp -o /tmp/main && /tmp/main < input.txt`
Java	`Main.java`	`javac -d /tmp Main.java && java -XX:+UseSerialGC -Xms16m -Xmx128m -cp /tmp Main < input.txt`

5. Security & Isolation Matrix

The platform implements multi-layer defense against hostile code execution:

- Filesystem Immutability (-v ...:/workspace:ro): The host working directory is mounted as read-only. Candidate code cannot alter, create, or delete source files on the host filesystem.
- Ephemeral Memory Compilation: All intermediate binaries (/tmp/main) and compiled classes (/tmp/Main.class) are written strictly to the container's RAM-backed /tmp directory.
- Complete Network Isolation (--network none): Inbound and outbound sockets, port binding, and DNS lookups are disabled to prevent remote data exfiltration or reverse shells.
- Resource Clamping:
 - Memory: Capped at 256MB to prevent out-of-memory host crashes.
 - CPU Allocation: Set to 1.5 cores to allow fast compilation while preventing infinite CPU consumption.
 - Process Limiting (--pids-limit 64): Halts recursive process spawning (fork bombs).
 - Privilege Revocation (--cap-drop ALL, --security-opt no-new-privileges:true): Strips all root Linux kernel capabilities and prevents privilege escalation.

6. API Reference & Payloads

1. Start Assessment

- Endpoint: `POST /api/start-assessment`
- Request:

```
{
  "userId": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11"
}
```

- Response:

```
{
  "status": "success",
  "session": {
    "sessionId": 19,
    "totalScore": 0,
    "maxScore": 300,
    "status": "In Progress"
  },
  "questions": [
    {
      "questionId": 7,
      "title": "Reverse Words in Sentence",
      "sampleTestCases": [
        { "testCaseId": 49, "inputData": "the sky is blue", "expectedOutput": "blue is sky the" }
      ]
    }
  ]
}
```

2. Run Code (Sample Evaluation)

- Endpoint: `POST /api/run-code`
- Request:

```
{
  "questionId": 7,
  "language": "java",
  "code": "import java.util.Scanner;\n\npublic class Main {\n    public static void main(String[]\nargs) {\n    Scanner scanner = new Scanner(System.in);\n    if (scanner.hasNextLine()) {\nString[] words = scanner.nextLine().trim().split(\"\\\\\\\\s+\\\\\");\n    StringBuilder sb = new\nStringBuilder();\n    for (int i = words.length - 1; i >= 0; i--) {\n    sb.append(words[i]).append(\" \");\n    }\n    System.out.print(sb.toString().trim());\n    }\n    }\n}"
```

- Response:

```
{
  "status": "success",
  "questionId": 7,
  "summary": {
    "totalTestCases": 3,
    "passedCount": 3,
    "failedCount": 0,
    "allPassed": true
  },
  "sampleTestCases": [
```

```

    {
      "testCaseId": 49,
      "inputData": "the sky is blue",
      "expectedOutput": "blue is sky the",
      "actualOutput": "blue is sky the",
      "status": "PASSED",
      "executionTimeMs": 312
    }
  ]
}

```

3. Submit Code (Full Evaluation)

- Endpoint: `POST /api/submit-code`
- Request:

```

{
  "userId": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11",
  "sessionId": 19,
  "questionId": 7,
  "language": "java",
  "code": "..."
}

```

- Response:

```

{
  "status": "success",
  "submissionId": 38,
  "sessionId": 19,
  "questionId": 7,
  "result": {
    "status": "Accepted",
    "score": 100,
    "totalMarks": 100,
    "passSummary": "8/8 Passed",
    "passedTestCases": 8,
    "totalTestCases": 8,
    "executionTimeMs": 2480
  }
}

```

4. Complete Assessment

- Endpoint: `POST /api/complete-assessment`
- Request:

```

{
  "userId": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11",
  "sessionId": 19
}

```

- Response:

```

{
  "status": "success",
  "message": "Assessment completed successfully",
  "session": {
    "sessionId": 19,
    "userId": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11",
    "totalScore": 300,
  }
}

```

```

    "maxScore": 300,
    "questionsCompleted": 3,
    "status": "Completed"
  }
}

```

7. Database Entity Relationships

- ``education.user_coding_sessions``: Tracks candidate exam attempts. Holds session state (``In Progress``, ``Completed``), final calculated marks, and question completion counts.
- ``education.coding_questions``: Master catalog of coding challenges, titles, descriptions, constraints, memory limits, and supported language configurations.
- ``education.coding_test_cases``: Test vectors mapped to specific questions via ``question_id``. Contains ``input_data``, ``expected_output``, and boolean flags (``is_hidden``, ``is_sample``).
- ``education.coding_submissions``: Granular log of every candidate submission. Records snapshot code, execution duration, individual test pass/fail results, and computed points.

8. Installation & Setup

Prerequisites

- Node.js: ``v18.0.0+``
- PostgreSQL: ``v14.0+``
- Docker Engine: Installed and running with access to the following base images:
- ``kiranvayvora/edusaas-python:latest``
- ``kiranvayvora/edusaas-cpp:latest``
- ``kiranvayvora/edusaas-java:latest``

Step-by-Step Installation

18. Clone the repository and install dependencies:

```

git clone <repo-url>
cd edusaas-compiler
npm install

```

19. Configure Environment Variables (`` .env ``):

```

PORT=6000
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=your_password
DB_NAME=edusaas

```

20. Initialize the Database Schema:

```

psql -U postgres -d edusaas -f sql/schema.sql

```

21. Pull Compiler Docker Images:

```

docker pull kiranvayvora/edusaas-python:latest
docker pull kiranvayvora/edusaas-cpp:latest
docker pull kiranvayvora/edusaas-java:latest

```

22. Start the Compiler Server:

```
npm start
```